

descSPIM-Galaxy: Python Control and Image processing Instructions

1. Environment creation

- 1-1. Application Installation
 - 1-1-1. Installation of Python
 - 1-1-2. Installation of Anaconda
 - 1-1-3. Installation of ThorCam™ software
 - 1-1-5. Installation of Kinesis® software
- 1-2. Environment Setup
 - 1-2-1. Creation of a new environment
 - 1-2-2. Installation of required packages
 - 1-2-3. Environment verification
- 1-3. Initial setup procedure in Micro-Manager
- 1-4. Changing RAM settings in ImageJ

2. Execute Python code

- 2-1. Image acquisition
- 2-2. Image preprocessing
- 2-3. Image processing
- 2-4. Nonlinear registration (SyN)

3. Troubleshooting

- 3-1. Errors related to permission
- 3-2. USB connection failure
- 3-3. Failure to load configuration file (Micro-Manager)
- 3-4. Backslash error in file paths
- 3-5. Insufficient memory in Jupyter Notebook
- 3-6. Insufficient RAM or system resources
- 3-7. Forgetting to close control software

1. Environment creation

1-1. Application Installation

Install the required environment from the following URLs.

1-1-1. Installation of Python

<https://www.python.org/>

Download the installer and follow the instructions to install Python. Select "Add Python to PATH" during installation and add Python to the PATH environment variable.

Currently, this PC is using Python 3.12.3 (64-bit). Different versions or architectures may cause unexpected errors.

1-1-2. Installation of Anaconda

<https://www.anaconda.com/download>

Run the downloaded installer. Follow the instructions provided by the installer. Usually, the default settings are fine. If you select the option "Add Anaconda to my PATH environment variable", you can use Anaconda from the command line (Windows).

We recommend downloading the latest version of Anaconda Distribution. Currently on this PC, Anaconda CLI, version 1.12.3 is used. Search and open Anaconda prompt from home.

```
jupyter notebook
```

Execute the above command and jupyter notebook should be up and running.

1-1-3. Installation of Micro-Manger

<https://download.micro-manager.org/nightly/2.0/Windows/>

Install the latest version from the above URL. At this time, if the version date is included in the installation, such as "C:\Program Files\Micro-Manager-2.0.231214", change the name to "C:\Program Files\Micro-Manager-2.0".

Next, from the Start menu, select "Control Panel" -> "System" -> "Advanced System Settings" -> "Environment Variables" -> "Path" from "System Environment Variables" -> "Edit" -> "New" -> add the path of the directory where Micro-Manager is installed (e.g. C:\Program Files\Micro-Manager-2.0)

When editing is complete, click "OK" on all windows to close them. Then restart the PC.

1-1-4. Installation of ThorCam (application for Thorlabs cameras)

https://www.thorlabs.co.jp/software_pages/ViewSoftwarePage.cfm?Code=ThorCam

Install ThorCam Software for 64-Bit Windows from the above URL. Follow the installation guide. Use version 3.7.0.6.

1-1-5. Installation of Kinesis® software (Thorlabs motor application)

https://www.thorlabs.co.jp/newgrouppage9.cfm?objectgroup_id=10285

Install Kinesis® software from the above URL. Follow the installation guide. Use version 1.14.52

1-2. Environment Setting

1-2-1. Creating a new environment

Launch Anaconda prompt and execute the following commands, this will create and activate a new environment (myenv).

```
python -m venv myenv
myenv\Scripts\activate
```

Confirm that the environment is activated. You should see the following in your terminal window:

```
(myenv) C:\Users\username>
```

1-2-2. Install the required packages

The following packages are required to run **all scripts and GUIs** shared so far

(including descSPIM control GUIs, PSF/FWHM analysis tools, stacking/averaging, hyperspectral preprocessing, and imaging utilities).

```
pip install jupyter
# Launch and manage .ipynb notebooks if needed

pip install numpy==1.26.4
pip install pandas==2.2.3
pip install opencv-python==4.11.0.86
pip install tifffile==2025.1.10
pip install pyserial==3.5
pip install pythonnet==3.0.5
pip install pymmcore-plus==0.13.1
```

```
pip install pillow==10.4.0
```

Optional but recommended for full compatibility with advanced scripts:

```
pip install matplotlib==3.10.0
pip install scipy==1.15.2
pip install scikit-image==0.25.2
pip install numba==0.61.2
pip install napari==0.6.0
pip install pyqt5==5.15.11
pip install ttkbootstrap==1.10.1 # if GUI theming is used instead of
azure.tcl
```

1-2-3. Environment verification

After installation, confirm versions with:

```
python -V
pip list
```

1-3. Initial setup procedure in Micro-Manager

1-3-2. Install ThorlabKinesis device adapter

<https://github.com/micro-manager/mmdev-ThorlabsKinesis/releases>

Download mmdev-ThorlabsKinesis-20230207.71-x64.zip of v20230207.71 from the above URL. Unzip the zip file and copy "C:\Users\username\Downloads\mmdev-ThorlabsKinesis-20230207.71-x64\mmdev-ThorlabsKinesis-20230207.71-x64\mmgr_dal_ThorlabsKinesis.dll" to "C:\Program Files\Micro-Manager Move "C:\Program Files\Micro-Manager-Micro-Manager-2.0".

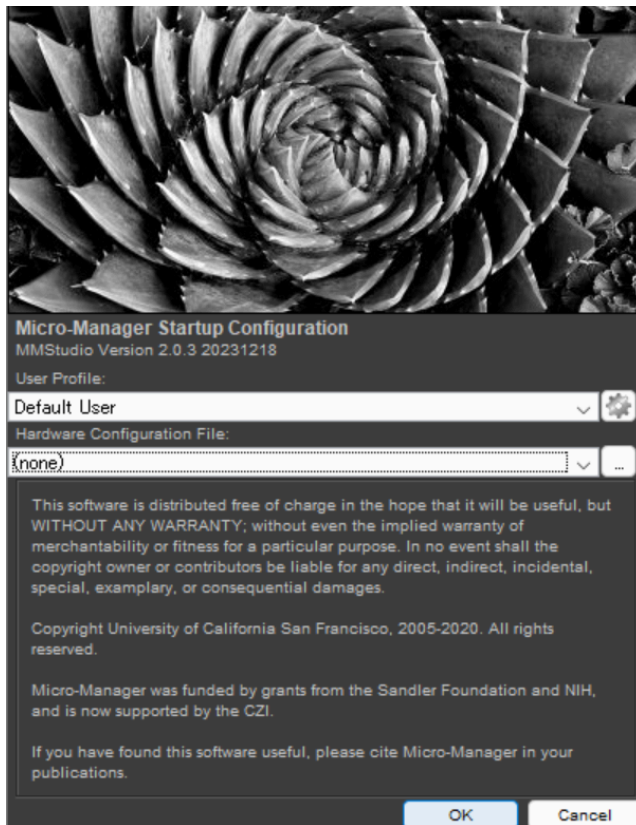
Repeat these steps for mmcore installed in your environment

"C:\Users\username\AppData\Local\pymmcore-plus\pymmcore-plus\mm\Micro-Manager_2.0.3_20240306\"

1-3-3. Create Configuration File

Launch Micro-Manager from the Start menu.

When Micro-Manager starts up, select "none" in Hardware Configuration File without changing User Profile and click OK to start Micro-Manager.



After Micro-Manager is launched, select device → Hardware Configuration Wizard from the menu bar.

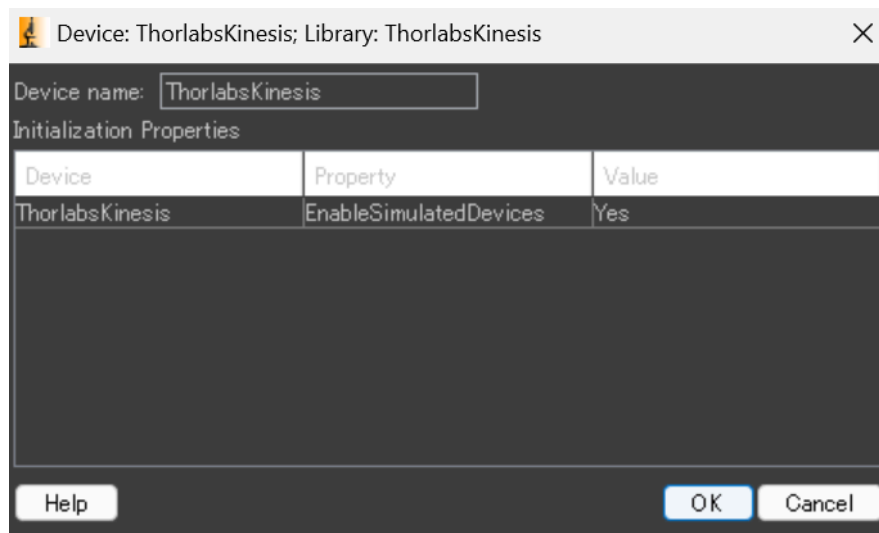
At this time, make sure that KDC101 motors (2 units) are connected to the PC. Poor connection may cause errors.

<Creating the ThorlabKinesis configuration file>

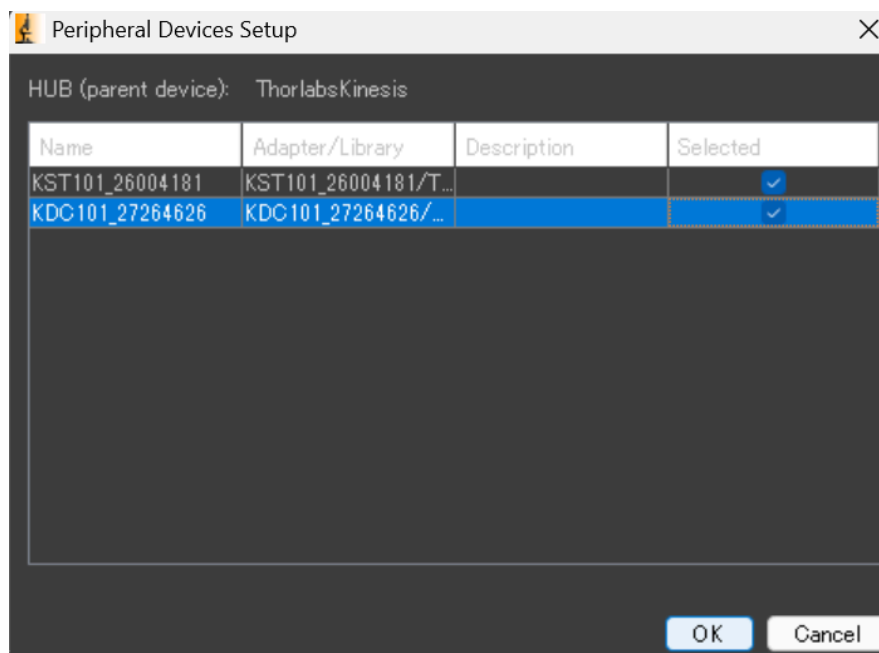
Launch Micro-Manager again from the Start menu. After launching Micro-Manager, do not change the User Profile, select none in Hardware Configuration File, and click OK to launch Micro-Manager.

Select devices → Hardware Configuration Wizard from the menu bar, and from Available Devices:, select ThorlabKinesis → ThorlabsKinesis: Thorlabs Kinesis Hub, Click add.

In the next window, the Device Name should remain ThorlabsKinesis.

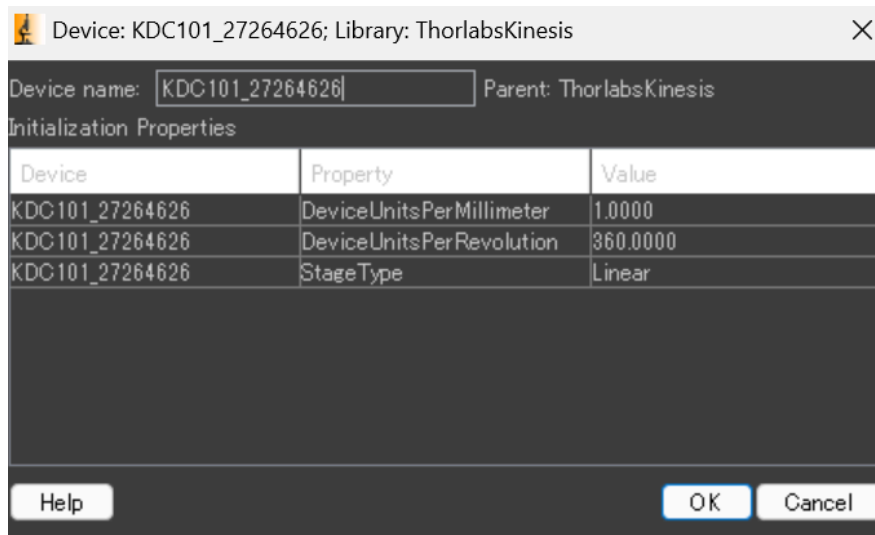


Select OK and proceed.

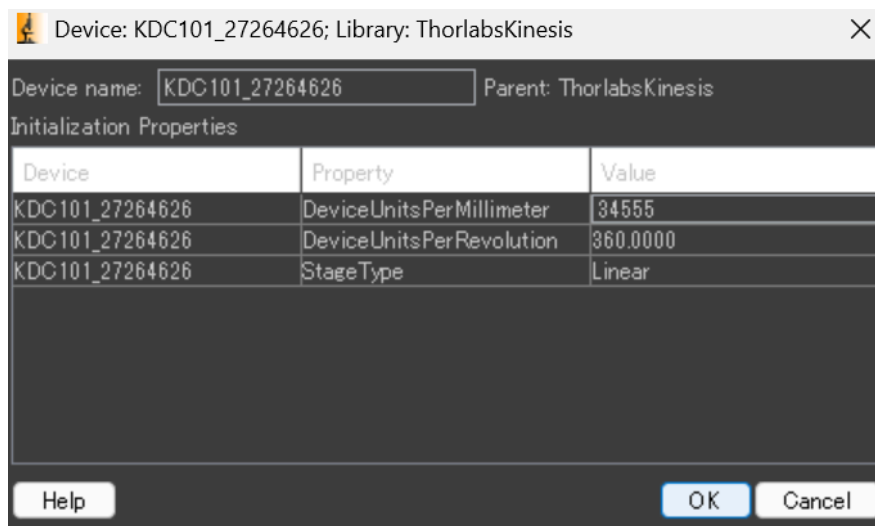


Confirm that two motors are recognized. After confirming, select all the checkboxes under Selected and click OK.

Check the Device name, which shows the serial number after “KDC_101_”. If it is the number of the stage on the camera side, set the device name to “cameraStage”. Similarly, if it is the sample side, set it to “sampleStage”.



After changing the Device name, change the value of “DeviceUnitsPerMillimeter” in the first line from 1.0000 to 34555 and the “StageType” to “Linear”.



※The value given to "Value" is specific to the device unit of each motor model, and this value is used in this descSPIM because a Z825B motor is used. See below for the calculation method.

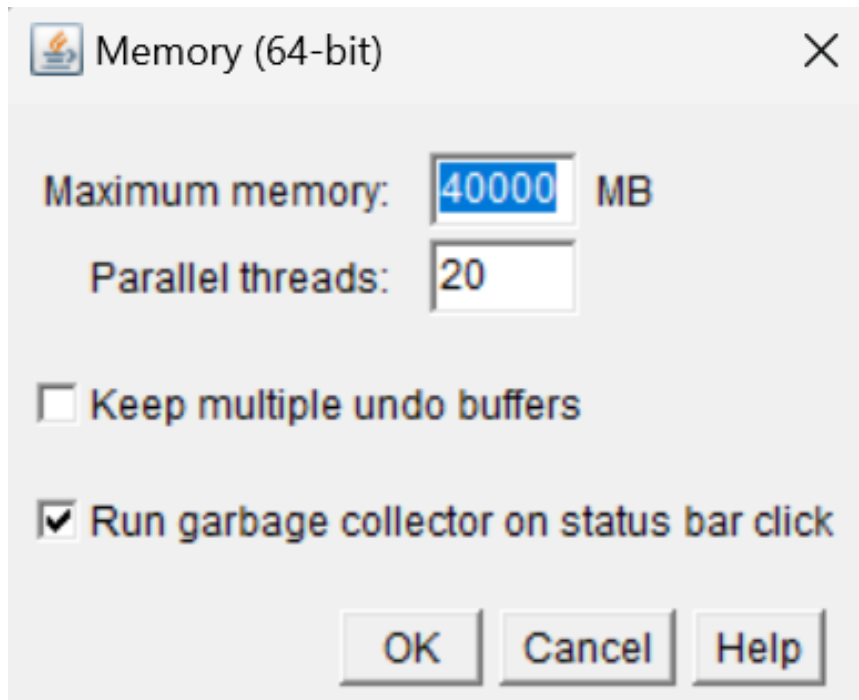
<https://github.com/micro-manager/mmddev-ThorlabsKinesis>

After, proceed to the same window as before and specify the file name and file path. Click "Finish" to confirm that the configuration file exists in the specified path, and then close Micro-Manager.

1-4. Changing RAM settings in ImageJ

Since Micro-Manager uses the RAM as a temporary storage location for captured images, it is essential to change the setting when acquiring large image stacks. The procedure is described below.

Launch Micro-Manager, then launch Edit → Options → Memory & Threads from the imageJ menu bar.



In the above window, change the value in the Maximum memory box. The number to be added here depends on the number of images acquired in the stack. 1000 images require about 25 GB of storage. Therefore, for this installation, it is recommended that Maximum memory be set to 40000 MB or more.

When acquiring more than 500 images for the first time, it is recommended to open the Task Manager from the Home menu, go to Performance -> Memory, and observe the memory usage while acquiring to make sure that Micro-Manager RAM usage does not exceed its capacity.

See: https://micro-manager.org/Micro-Manager_Configuration_Guide

2. Executing Python Code

Note:

Several programs are provided in both Spyder (.py) and Jupyter Notebook (.ipynb) formats. Running the .py versions in Spyder is recommended for long-term stability and reliable local execution.

2-1. Image acquisition

Before running `Snapscan_Multiex_3D_GUI.py`, update local paths and ensure they are properly recognized by your environment:

Set the imec HSI Snapscan Python API path
In the script header, modify:

```
SNAPSCAN_API_PATH = r"C:/imec/HSI Snapscan v2.4.13.0/python_apis"
```

1. to match your actual Snapscan Python API installation path.
Ensure that this directory is accessible to Python (either via `sys.path.insert()` as written in the script, or by adding it to PATH/PYTHONPATH).

Specify your own camera XML and Micro-Manager configuration files

Update the GUI defaults (or browse for them when launching the GUI):

```
self.v_xml_camera =  
tk.StringVar(value=r"D:/your_folder/your_camera.xml")  
self.v_cfg_mm      =  
tk.StringVar(value=r"D:/your_folder/your_micromanager.cfg")
```

2. These files correspond to your camera configuration (XML) and Micro-Manager system configuration (.cfg).

Confirm the Thorlabs Kinesis® DLL path

If installed in a non-default directory, adjust:

```
KINESIS_DLL_ROOT = r"C:/Program Files/Thorlabs/Kinesis"
```

3. so that the script can load DLLs correctly via `clr.AddReference(...)`.

(Optional) Prefill device ports and serial numbers

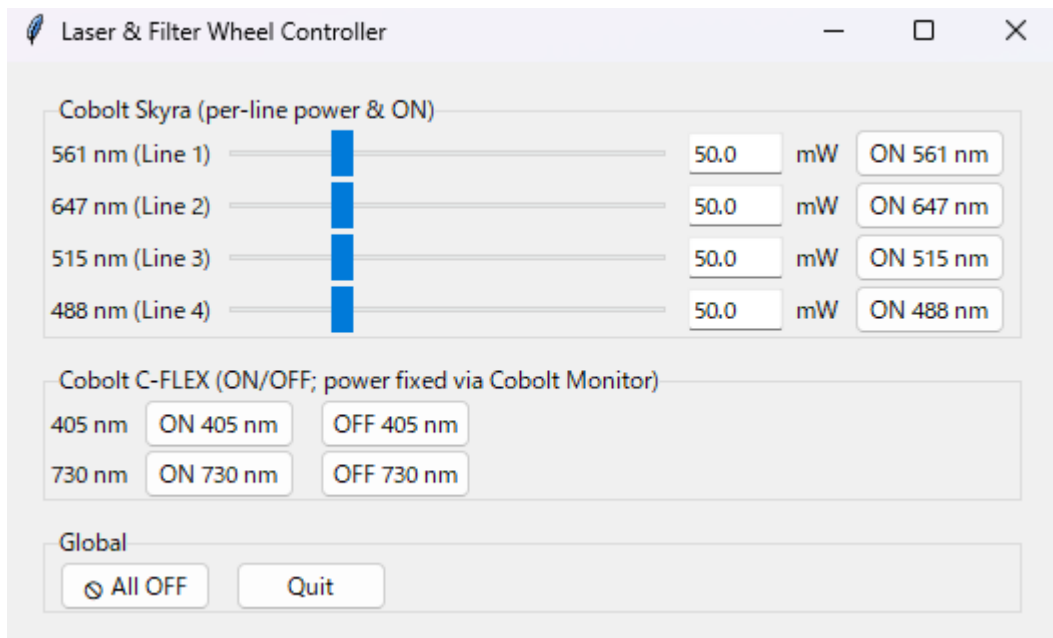
For repeated use, you may preset:

```
self.v_fw_sn.set("YOUR_SERIAL")  
self.v_port_cflex_405.set("COMx")  
self.v_port_skyra.set("COMy")
```

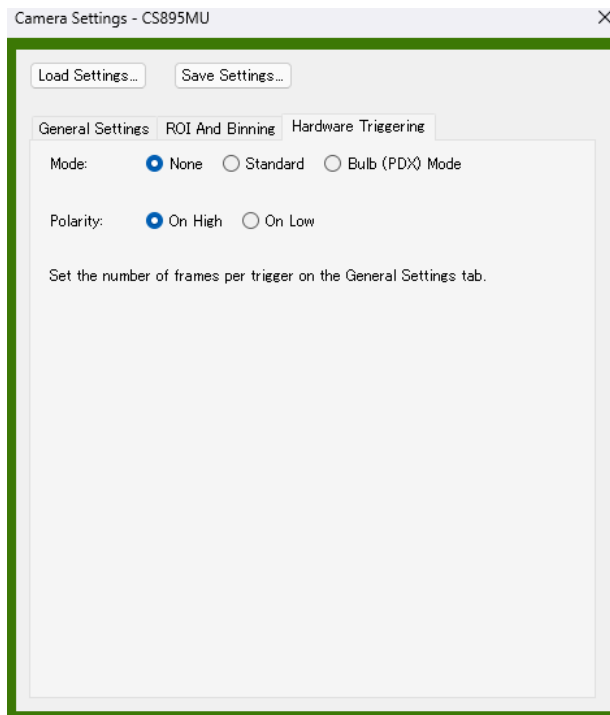
4. These can also be edited directly in the GUI.

After verifying these paths and settings, proceed with imaging as follows:

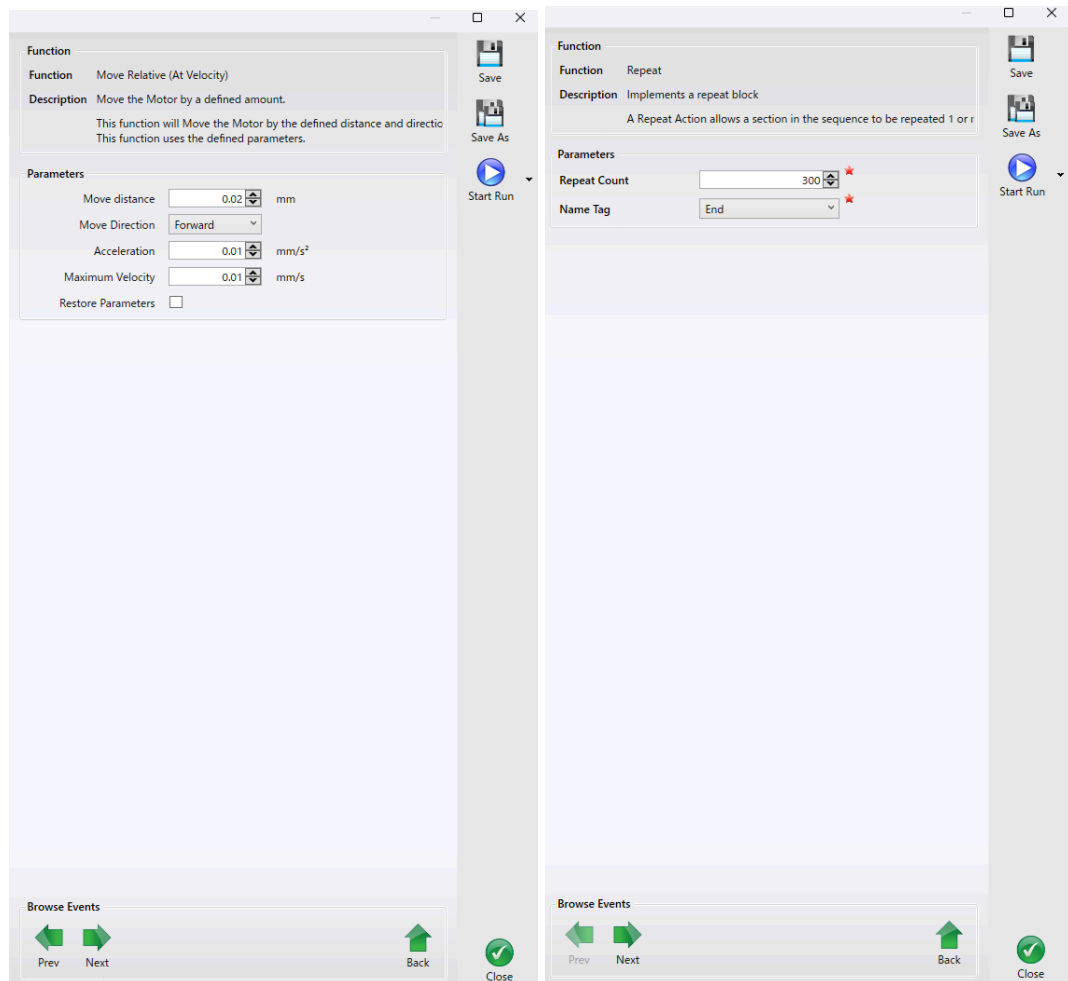
- Prepare single-stained samples (RUNX2–Alexa 647, aSMA–Cy5.5, TNNI3–Cy3, TUBB3–Alexa 594).
Adjust laser power and exposure time to match multiplex imaging conditions.
Launch HSI Snapscan, select the corresponding excitation line, locate the field of strongest fluorescence, crop the region, and export the mean spectrum as a CSV reference file.
- Start the descSPIM-Galaxy system. Launch Kinesis® and ThorCam®, set the velocity ratio between the sample and camera stages to 1 : 0.342.
Insert a 45° silver mirror, mount the cleared sample, and enable the laser combiner.
Using ThorCam live view, center the focal plane and set exposure to 300 ms for all channels.
- Open Spyder and run Laser and Filter Wheel Controller_GUI.py (or the notebook version).



This GUI synchronizes the laser combiner and motorized filter wheel.
In ThorCam, open *Settings* → *Hardware Triggering* and set *Mode* = *Standard*, *Frames per Trigger* = 1, *Polarity* = *On High*.



- Confirm in the sequence file that *Move Distance* = $20\ \mu\text{m}$ and *Repeat Count* = 300 (total 6 mm).



Press REC in ThorCam, then execute RUN in Kinesis®.

Select 8-bit mode to minimize delays. After each stack, the stage returns automatically; laser and filter switching are handled by the GUI.

- After CMOS imaging, record `sample_start_Z` and `camera_start_Z`.
Remove the mirror and close all applications.
Move the sample to the center and record `sample_focus_Z`.
Reopen HSI Snapscan, refocus for each excitation, and record `camera_focus_Z`.

Compute:

$$\text{camera_start_Z} = \text{camera_focus_Z} - 0.342 \times (\text{sample_focus_Z} - \text{sample_start_Z})$$

- Then open Spyder and run `Snapscan_Multiex_3D_GUI.py` (or the notebook version).

Snapscan Multi-Excitation 3D Imaging

Paths

SAVE_ROOT

XML_CAMERA

CFG_MM

Acquisition

IT_MS (loop count N) Integration (ms)

Sample step (μm) Camera step (μm) = sample × 0.342

Start positions for loop 1 (WRITE YOUR MEASURED FOCUS HERE)

ex647 sample Z (μm)	12716.92	ex647 camera Z (μm)	19558.64
ex561 sample Z (μm)	12716.92	ex561 camera Z (μm)	19558.64
ex405 sample Z (μm)	12716.92	ex405 camera Z (μm)	19558.64

Devices & Ports

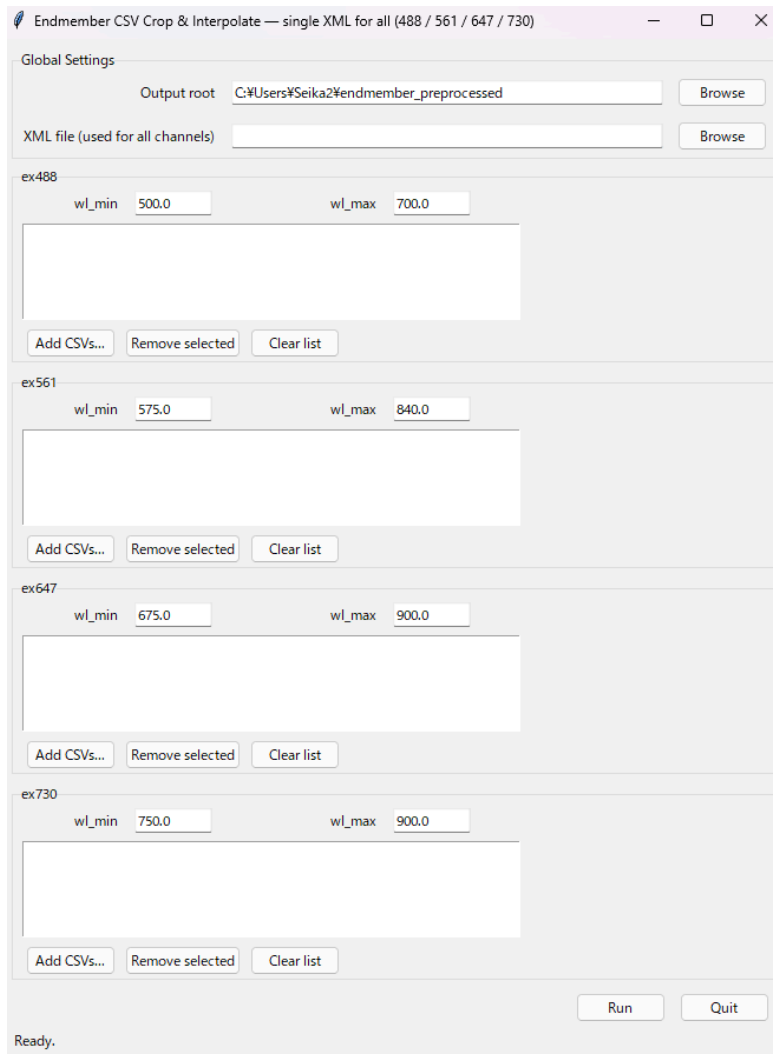
KCube FW SN C-FLEX (405) COM

Skyra COM

Enter each **camera_start_Z** and click Start. Hyperspectral 3-D acquisition proceeds automatically with 20 μm steps and 300 ms exposure. Approximate imaging time: 1.5 h (CMOS), 45 h (hyperspectral).

2-2. Image preprocessing

- Open Spyder and run preprocess_endmember_gui_single_xml.py.



In the GUI, select the XML configuration file and add the measured reference spectra CSVs:

- TNNI3–Cy3 and TUBB3–Alexa 594 (for ex561)
- RUNX2–Alexa 647 and aSMA–Cy5.5 (for ex647; use “aSMA” instead of “αSMA” to avoid path errors).

Run to generate interpolated CSV files such as

`Interp_crop_<label>_loopX_ex<wl>nm.csv`.

- Next, open `Endmember_and_BandExtraction_Combined.ipynb` in Jupyter Notebook (or run the integrated Python script if available).
Select a representative loop TIFF file, specify the output directory, and adjust band ranges if needed (e.g., ex561 575–750 nm; ex647 675–850 nm).
Click Run to automatically process all matching TIFF files in the same folder.
Results are saved as `Extracted_*.tif` under corresponding `Extracted_ex{wl}` directories.

Although both formats are available, the standalone `.py` scripts are recommended for reproducibility and independence from notebook dependencies.

2-3. Image processing

- Open Unmix_GUI_Multi_ex.ipynb in Jupyter Notebook.
Set the input directories (e.g., `Extracted_loopX_561nm_object_300ms.tif`, `Extracted_loopX_647nm_object_300ms.tif`) and the corresponding endmember CSVs.
Select the unmixing algorithm (NNLS or FCLSU) and run to generate unmixed TIFF outputs for each excitation.
Example output files include:
`Cy3_Extracted_loop12_561nm_object_300ms.tif`,
`Runx2_Extracted_loop12_647nm_object_300ms.tif`.
- Next, open Stack_and_Avg_GUI.py (or the `.ipynb` version if preferred).

The screenshot shows a GUI titled "Stack multipage (ex488/561/647/730) + Average & stack (ex405)". It has a "Global" section at the top with an "Output root" field set to "C:\Users\Seika2\multipage_output" and a "Browse" button. Below this are five sections for different excitations: "ex488 — Stack unmixed results to multipage", "ex561 — Stack unmixed results to multipage", "ex647 — Stack unmixed results to multipage", "ex730 — Stack unmixed results to multipage", and "ex405 — Average first 1–5 bands by nm". Each of the first four sections has a "Source folder" field with a "Browse" button, a "Loops (N)" field set to 300, a "Label filter (regex, optional)" field, and a checkbox "Also save individual per-loop files under /Singles". The "ex405" section has a "Source folder (loop TIFF stacks)" field with a "Browse" button, a "Loops (N)" field set to 300, an "nm_min (≥470)" field set to 470.0, a "First K bands within [nm ≥ nm_min] (1–5)" field set to 5, an "XML (optional; else sidecar)" field with a "Browse" button, and a checked checkbox "Also save per-loop averaged images". At the bottom right are "Run" and "Quit" buttons. The status bar at the bottom left says "Ready."

Specify the output root folder and input directories for ex561, ex647, and ex405. For ex561, use files starting with "Cy3_" or "A594_"; for ex647, use the corresponding label names.

For ex405, select the raw cube folder and average channels 1–5.

Running once will produce stack files such as

`Cy3_loop1-N_stack.tif`, `A594_loop1-N_stack.tif`, and `ex405_avg_stack.tif`.

- Finally, open `tiff_Downsample_and_ECC_Registration_GUI.ipynb`, or alternatively run `Downsample TIFF files from 3.45 µm to 5.5 µm.py` and `ECC Registration.py` in Spyder.

Section 1 performs downsampling of CMOS TIFF stacks (e.g., POPO-1, TAG-1-FITC, CD31-D755) from 3.45 µm/px to 5.5 µm/px and generates `*_CMOS_binned.tif`.

Section 2 performs ECC registration using `ex405_avg_stack.tif` as the fixed reference and produces rigidly aligned outputs `*_CMOS_rigid_affine.tif`.

Each process launches a GUI for interactive file selection and execution.

2-4. Nonlinear registration (SyN)

- Upload the fixed reference image `ex405_avg_stack.tif` and all rigid/affine-registered CMOS stacks (e.g., `POP01_CMOS_rigid_affine.tif`, `TAG1_CMOS_rigid_affine.tif`, `CD31_CMOS_rigid_affine.tif`) to your Linux workstation or analysis server.

This environment is running Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-141-generic x86_64), which provides stable compatibility for ANTs (Advanced Normalization Tools) and GUI-based execution.

- Open Spyder and run `ANTS_Registration_GUI.py` (or the Jupyter Notebook version, if available).
In the GUI, specify:
 - Fixed image: `ex405_avg_stack.tif`
 - Moving images: each rigid/affine-registered stack to be aligned (e.g., POPO1, TAG1, CD31)
 - Output directory: destination folder for saving registered results

Select the SyN (Symmetric Normalization) option for nonlinear registration.

This process performs voxel-wise deformation correction to precisely align all channels to the fixed reference stack.

After execution, the following warped outputs are generated:

`POP01_CMOS_SyN.tif`, `TAG1_CMOS_SyN.tif`, and `CD31_CMOS_SyN.tif`.

Verify alignment quality visually in napari or ImageJ/Fiji by overlaying each SyN-registered image with `ex405_avg_stack.tif`.

If necessary, adjust regularization or iteration parameters in the GUI and rerun until optimal spatial correspondence is achieved.

3. Troubleshooting

3-1. Errors related to permission

Even if the environment is configured correctly, unexpected permission errors may occur depending on the initial Windows user settings.

If a file cannot be executed or saved, right-click it and select Properties → General, then ensure that “Read-only” is unchecked.

If the problem persists, try running Spyder or Jupyter Notebook as an administrator.

3-2. USB connection failure

If an error such as “device not found” appears, the hardware (e.g., Kinesis controller, camera, or serial device) may not be recognized.

Check that all USB adapters and COM ports are properly connected. Reconnecting or restarting the Kinesis and ThorCam applications often resolves the issue.

3-3. Failure to load configuration file (Micro-Manager)

If an error such as “failed to load TSIcam” appears when launching Micro-Manager, the configuration file (.cfg) may not be loaded correctly.

Edit the environment variables to include the installation directory of Micro-Manager:

1. Open Control Panel → System and Security → System → Advanced system settings.
2. Click Environment Variables.
3. Under System variables, select Path → Edit → New.
4. Add the directory where Micro-Manager is installed (e.g., `C:/Program Files/Micro-Manager-2.0`).
5. Click OK to close all windows and apply the change.

Restart Micro-Manager and confirm that the configuration file loads normally.

3-4. Backslash error in file paths

When copying paths from Windows, backslashes (\) may appear as in `C:\Program Files\Micro-Manager-2.0`.

In Python scripts, replace them with forward slashes (/), i.e. `C:/Program Files/Micro-Manager-2.0`, to avoid escape-sequence errors.

3-5. Insufficient memory in Jupyter Notebook

When using Jupyter Notebook, a Kernel crash may occur due to memory exhaustion.

You can modify Jupyter’s memory allocation limit:

Navigate to your Jupyter config directory:

```
cd C:\Users\<username>\.jupyter
```

1. Open `jupyter_notebook_config.py` in a text editor:

```
notepad jupyter_notebook_config.py
```

2. Add the following line:

```
c.NotebookApp.max_mem_rate = 0.9
```

3. This allows the notebook to use up to 90% of the available RAM.
Save and restart Jupyter Notebook:

```
jupyter notebook
```

4. If the error persists, terminate unnecessary background processes via Task Manager (Ctrl + Shift + Esc) to free memory.
Check the Jupyter log for details:

```
jupyter --paths  
cd ~/.jupyter  
notepad jupyter.log
```

3-6. Insufficient RAM or system resources

If your PC has less than 64 GB of RAM, or other heavy applications are running simultaneously, memory shortages may prevent large-stack processing (e.g., FWHM analysis, ECC registration, SyN deformation).

To check and optimize memory:

- Check RAM:
Open Task Manager → Performance → Memory to confirm total and available RAM.
- Terminate unnecessary applications:
End unnecessary background programs.
- Manage startup programs:
Disable unused apps under the Startup tab.
- Delete temporary files:
Use Disk Cleanup to free disk space.

For large hyperspectral or 3D stack datasets, it is strongly recommended to use a workstation with ≥64 GB RAM for stable performance.

3-7. Forgetting to close control software

The hyperspectral camera and laser source are sometimes operated using their dedicated control applications — HSI SNAPSCAN® v2.4.13.0 for the hyperspectral camera and Cobolt Monitor® v6.1.4.0 for the laser source.

Both devices can also be controlled directly from the custom Python GUI.

However, if one of these official control applications (HSI SNAPSCAN or Cobolt Monitor) remains open while attempting to connect through the Python GUI, the program will fail to detect the hardware and an error such as “device not found” will occur.

Before launching one application, make sure the other is completely closed to avoid connection conflicts.